Lifelong Learning for an Origami Robot: Supervised Learning of Target Adaptations

E. De Vroey

***Abstract*—Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.**

I. Introduction

II. Background

A. Lifelong Learning

B. Kresling Origami

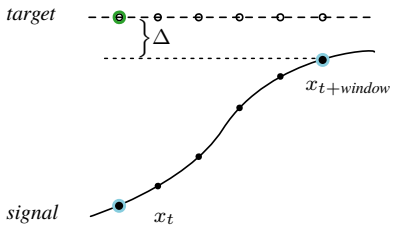
III. Methodology

A. Target Adaptations

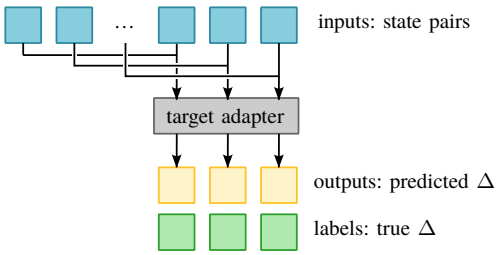
The method proposed in this paper aims to provide a way of improving a given controller's performance as well as fortifying it against evolving system dynamics *without* having access to the inner workings of the controller itself. In other words, as long as the fundamental assumptions of a controller are satisfied—i.e. it requires a state and a target as input and yields a control action—the method can remain completely agnostic to the specific controller being used. In addition, the aim is not to replace the controller entirely but to introduce an additional module to the control scheme that enables lifelong robustness to evolving system dynamics. The advantage of such a method is that it could be applied to any system as a *plug-and-play* module.

The manner in which this will be achieved is by *target adaptation*. An *adapter* module modifies the target used by the controller to improve its performance. Consider a scenario where a controller yields a control action u to reach a certain desired target state x_d. As the system dynamics evolve, suppose this action u is now insufficient to reach the target. In response, the adapter module *lies* to the controller about reaching x_d and instead provides it with $x_d + \Delta$, for which the controller yields a larger action u^* that *does* result in the system reaching state x_d.

The *adapter* module is implemented as a feedforward neural network. To train this model to learn suitable target adaptations, interactions of the controller with the system are recorded. The model is then provided with pairs of system states, spaced by a certain time window x_t and $x_{t+\text{window}}$, that are obtained from these recordings. From these pairs, the model infers the target it 'believes' the controller aims to reach. However, instead of predicting the target directly, the model learns the difference Δ between the target and the latter state in the pair (figs. 1a and 1b).



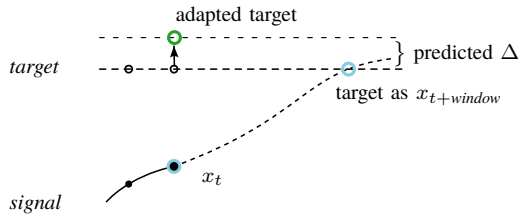
(a) The target adaptation problem: Based on the current x_t and the state some prediction window later $x_{t+window}$, the model learns to predict the target the controller is trying to reach by representing it as a Δ to be added to $x_{t+window}$.



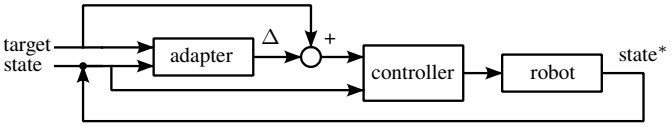
(b) The training process of the target adapter: recorded states spaced by a specified prediction window are paired up and provided as features to the adapter, which learns to predict a Δ, being the difference between the target x_d and state $x_{t+window}$.

Fig. 1: Target adaptation during the training phase

The model can be trained in a supervised manner (fig. 1b) and then deployed. During deployment, it is presented with the current state x_t but cannot be presented with $x_{t+window}$ (since this state is not known yet). Instead, the second state in the pair is set to the desired target. To the model, this represents a situation in which the desired target was reached from the current state within the specified time window. The idea is now that the trained model will produce a Δ that, when added to the latter state in the pair (i.e., the desired target), results in an adapted target (fig. 2a) for which the controller will produce a control action. In short, given a current state x_t and a future state $x_{t+window}$, the model learned to produce the desired target. Thus, when presented with x_t and $x_{t+window} = x_d$ (supposedly reached within the time window) it provides the (adapted) target that would *push* the controller toward that result. A full overview is presented in figs. 2a and 2b.



(a) Target adaptation during deployment: By supplying the model with the controller's target as $x_{t+window}$ it yields a Δ that can be added to the current target, which will supposedly result in the controller reaching the target within the specified window.



(b) Schematic overview of the full implementation: The adapter is used to offset the given target before presenting it to the controller, essentially *lying* to the controller in order to improve its performance.

Fig. 2: Target adaptation during deployment.

The reason for predicting the difference Δ between the state $x_{t+window}$ and the desired target, rather than a direct prediction of the target, can also be clarified now: By designing the model's output to be a Δ, the model can be initialized with parameters all close to zero (essentially leaving the desired target unchanged) and then steadily improve online as interactions of the controller and system are recorded, without the need for a pretraining phase.

B. Toy Problem

To explore the proposed method, a toy problem is considered first. The toy problem consists of a simulation of a simple 1D mass-spring-damper system (fig. 3) and an imperfectly tuned PID controller. The mass is suspended in a hanging position with gravity acting on it continuously.

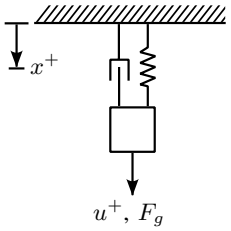


Fig. 3: Setup of the mass-spring-damper system in the toy problem with positive directions indicated.

At regular intervals, a random target position x_d is generated for the controller to reach and maintain. Each timestep, the adapter calculates the required Δ, and the controller computes the control action base on the current state and the adapted target. The system's response to this control input is simulated and the tuple $(x_0, u, x, x_d + \Delta)$ is added to the buffer. This buffer stores a limited number of tuples corresponding to a specified window of the most recent data. Once capacity is reached, the oldest element is removed at every timestep. At regular update intervals, the adapter is trained on the data in the buffer. It is important that the tuples in the training data always contain the target that was actually provided to the controller, i.e., $x_d + \Delta$, and not the *true* desired target x_d. As the simulation progresses, the parameters of the dynamic system are gradually altered to simulate system degradation. The pseudo algorithm for the implementation can be found in algorithm 1.

Algorithm 1: Toy problem target adaptation

Data: system state x_0, target x_d, adaption Δ, control action u
Initialize model: *adapter*;
Initialize buffer: *buffer* $\leftarrow []$;
for *each timestep* t_i **do**
  if *len(buffer) > threshold* **then**
    Remove the oldest element in the buffer;
  end
  if $t_i \mod target\_interval = 0$ **then**
    $x_d \leftarrow random\_target()$;
  end
  if $t_i \mod update\_interval = 0$ **then**
    (features, labels) $\leftarrow$ *prep_data(buffer, window)*;
    adapter.fit(features, labels);
  end
  $\Delta \leftarrow adapter.predict([x_0, x_d])$;
  $u \leftarrow controller.compute\_control(x_0, x_d + \Delta)$;
  Simulate the system response x;
  Update system parameters;
  Append $[x_0, u, x, x_d + \Delta]$ to *buffer*;
  $x_0 \leftarrow x$;
end

C. Preliminary Results

The toy problem was implemented in Python: simulating the system response using SciPy's `lsim`, and implementing the adapter in Keras with the PyTorch backend. The simulation ran for 60 seconds at 50 Hz. The target was changed Every 5 seconds, and the adapter was trained fully online, updating every 15 seconds. A prediction window of 10 timesteps was set for the adapter, meaning that it's input features during training consisted of $[x_t, x_{t+10}]$. The buffer's budget corresponded to the past 30 seconds of data. For reference, a second controller and system were also simulated, identical to the described simulation, but without the addition of an adapter.

cite

1) A case where the controller is too aggressive: For a first simulation, a combination of a system and controller was chosen where the controller's gains were tuned too high. In fig. 4, the result of the simulation can be observed. At the start, and due to the near-zero initialization of the adapter's parameters, the response signal of the system controlled by the adapted controller is indistinguishable from that of the reference controller. However, as soon as the adapter has trained on the buffer, the changes to the target become visible. The effects of the adaptation can be more clearly observed in fig. 5.

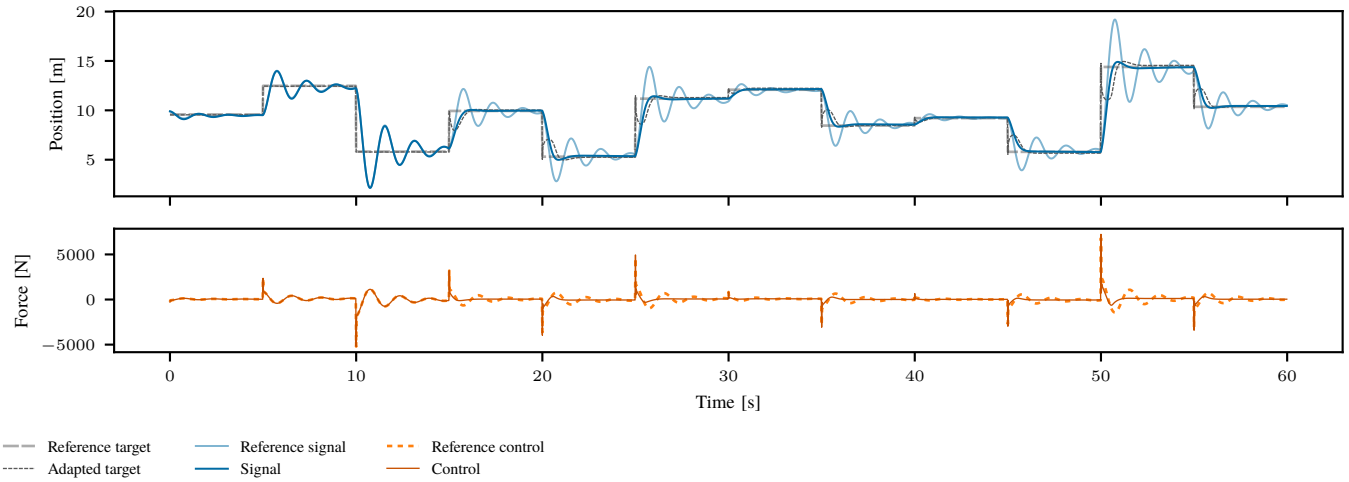


Fig. 4: The full minute simulation of the toy problem with an aggressive controller. Top: The system response x and target $x_d + \Delta$ of the adapted controller, alongside the response and target of the unadapted reference controller ($\Delta = 0$). Bottom: The resulting adapted control inputs from the controller receiving $x_d + \Delta$ alongside the unaltered reference control inputs. During the first 15 seconds (3 targets) the adapter has not received updates yet, and the signal is thus identical to the reference signal.

Since the reference controller is too aggressive, the reference signal overshoots and oscillates significantly before settling. In contrast, the adapter counters this by bringing the target closer to the current state. This is the logical result of the buffer containing data corresponding to the overshoots, where the adapter would often encounter training features for which the labeled target[1] x_d is somewhere in between x_t and x_{t+5}. Therefore, upon receiving the input feature $[x_t, x_d]$ (representing x_d being reached from x_t within 10 timesteps), it produces and adapted target in between x_t and x_d. As the state approaches the adapted target, the adapter starts producing targets closer to the reference target, guiding the system towards this desired state.

2) A case where the controller is under-responsive: For a second simulation, the system and controller were designed such that the controller was more *conservative* or *under-responsive*. Additionally, the signal produced by the reference controller has a clear steady-state error. The result of the simulation can be found in fig. 6.

Once more, the adapter is able to improve the controller's performance after just a single update. In fig. 7 it can be seen that the adapted target is adjusted away from the current state. This is because training features created from the data in the buffer frequently include state pairs x_t, x_{t+10} where the target x_d is positioned further from x_t than it is from x_{t+10}. As a result, the adapter learns to predict adaptations that would move x_d farther from x_t, making the controller more aggressive. Another effect of the adapter is the consistent offset applied to the target: In the training features, the adapter regularly encounters pairs x_t, $x_{t+10} = x_t$, which appear to represent the target state being reached, even though $x_{t+10} \neq x_d$. These features are the result of the steady-state error present in the signal, leading the adapter to learn to apply an offset to the target to counteract this.

[1]Rather, the labeled $\Delta = x_d - x_{t+10}$, but the explanation provided is equivalent and somewhat clearer.

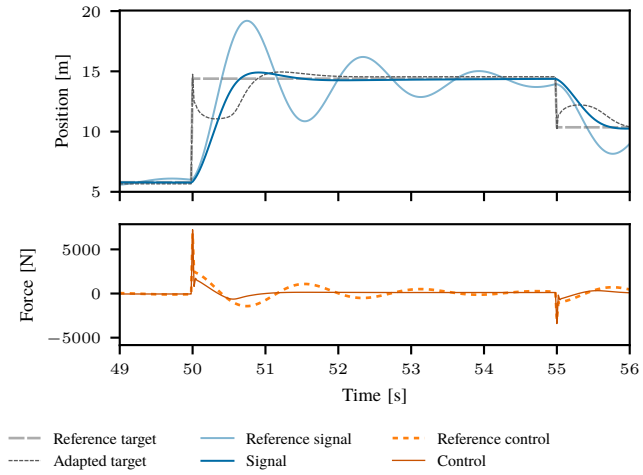


Fig. 5: A close-up of the second to last target in the one-minute simulation. The effects of the adapter can be more clearly observed: The overshoot (seen in the reference signal) is countered by bringing the target closer to the current state.

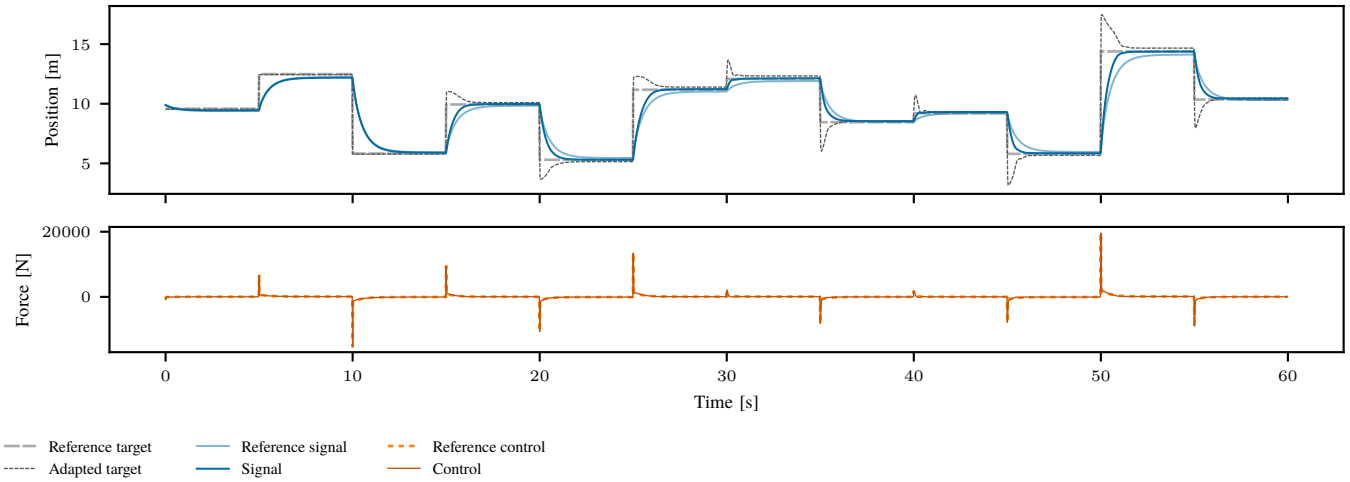


Fig. 6: The full minute simulation of the system with an uner-responsive controller. Top: The system response and target of the adapted controller, alongside the response and target of the unadapted reference controller. Bottom: The resulting adapted control inputs from the controller receiving the altered target alongside the unaltered reference control inputs.

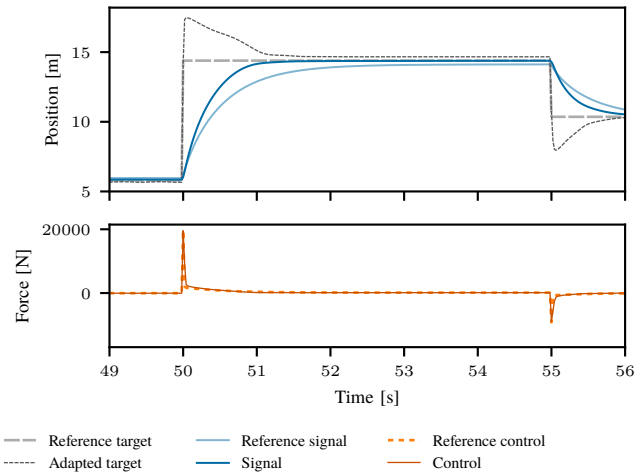


Fig. 7: A close-up of the second to last target in the one-minute simulation. Noteable observations: The target is initially moved further from the current state to 'pull' it towards the desired state in a shorter period. In addition, the adapted target is offset from the esired state as to counter the steady-state error seen in the reference signal.